

Enterprise Document Intelligence Chatbot

A Complete Build Guide — From Zero to a Shortlistable Infosys GenAI Project

Krishna Nirmal Gandhi | Computer Engineering, VIIT Pune

What You Are Building

A web application where a user uploads one or more enterprise PDF documents (contracts, HR policies, SOPs, compliance manuals) and then asks natural-language questions about them. The system reads the documents, understands their content using AI, and returns accurate, cited answers — all without the user needing to search manually.

5

Core components

6

Build phases

~2hrs

Azure API setup

Streamlit

Demo UI

System Architecture

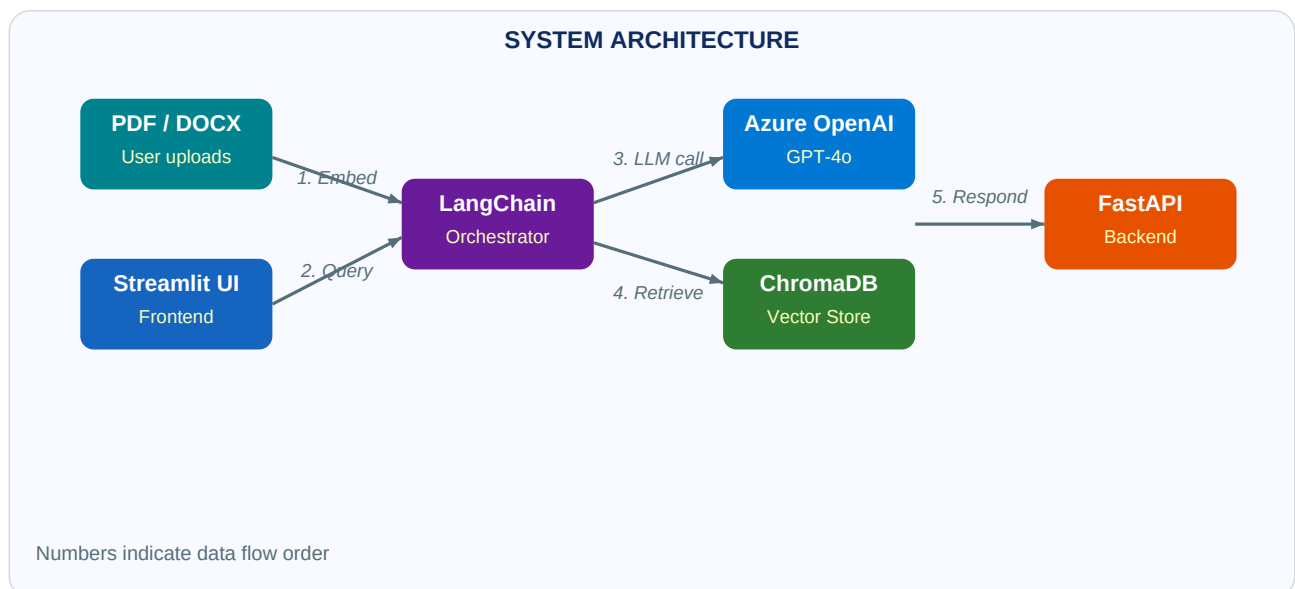


Figure 1 — Data flow from PDF upload to AI-generated answer

Build Roadmap



Table of Contents

1. What Is This & Why Infosys Cares

Enterprise context, RAG explained, why this is their #1 GenAI delivery

2. How It Works — The RAG Pipeline

Step-by-step: chunking, embedding, retrieval, generation

3. Tech Stack & Prerequisites

Azure OpenAI, LangChain, ChromaDB, Streamlit, FastAPI

4. Phase 1 — Environment Setup

Azure account, API keys, project structure, requirements.txt

5. Phase 2 — PDF Ingestion & Chunking

Loading PDFs, text extraction, chunking strategy, overlap

6. Phase 3 — Vector Store with ChromaDB

Embedding, storing, persisting, querying

7. Phase 4 — LangChain + Azure OpenAI Chain

QA chain, prompt template, source citations

8. Phase 5 — Streamlit Frontend

File uploader, chat UI, streaming responses, session state

9. Phase 6 — Deploy & GitHub Portfolio Setup

README, demo GIF, .env handling, optional FastAPI wrapper

10. Resume Bullet & Interview Q&A;

Exact bullet to add, 5 questions Infosys will ask you

1. What Is This & Why Infosys Cares

The Enterprise Document Intelligence Chatbot is a GenAI application that lets business users ask questions about their own documents in plain English. Instead of a lawyer manually searching a 200-page contract for a termination clause, they type: *'What are the termination conditions in this agreement?'* and get a precise, cited answer in two seconds.

Why Infosys Builds This for Clients

Industry Vertical	How Infosys Sells This
BFSI (Banking)	Loan agreement Q&A;, RBI compliance policy search, KYC document verification
Healthcare	Clinical protocol lookup, insurance policy Q&A;, patient record summarisation
Manufacturing	SOP retrieval, quality manual Q&A;, safety regulation search
Legal / Compliance	Contract review, clause extraction, regulatory change tracking
HR & Admin	Policy handbook Q&A;, offer letter parsing, leave policy retrieval

>> Why this project guarantees shortlisting

Infosys has a published GenAI service line called 'Infosys Topaz' — it is built almost entirely on RAG-based document intelligence delivered to enterprise clients. When you describe this project in an interview, you are describing their product roadmap. No other fresher applicant will walk in with this framing.

What RAG Means (Simply)

RAG stands for Retrieval-Augmented Generation. Instead of asking an LLM to answer from its training data alone (which is often wrong or outdated), RAG first searches your documents for the relevant passages, then feeds those passages to the LLM as context. The LLM then answers based on what it was just shown — not what it memorised.

Approach	How it works	Result
Without RAG	LLM answers from training data	Hallucinations, outdated info
With RAG	LLM answers from YOUR documents	Accurate, cited, up-to-date

2. How It Works — The RAG Pipeline

The entire system runs in two phases: an ingestion phase (run once per document) and a query phase (run every time a user asks a question).

Phase A — Ingestion (one-time per document)

Step 1 — Load: Read the PDF file and extract raw text from all pages using PyPDFLoader.

Step 2 — Chunk: Split the text into overlapping chunks of ~500 tokens. Overlap ensures context is not lost at chunk boundaries. Think of it as cutting a book into paragraphs that share two sentences with the next paragraph.

Step 3 — Embed: Convert each chunk into a vector (a list of ~1536 numbers) using Azure OpenAI's text-embedding-ada-002 model. Semantically similar text produces similar vectors.

Step 4 — Store: Save all vectors and their original text into ChromaDB, a local vector database that persists to disk. This is the 'knowledge base' the chatbot searches.

Phase B — Query (every user question)

Step 1 — Embed question: The user's question is embedded into a vector using the same model.

Step 2 — Retrieve: ChromaDB finds the top-K chunks whose vectors are most similar to the question vector. This is semantic search — it finds meaning-matches, not just keyword matches.

Step 3 — Augment: The retrieved chunks are inserted into a prompt template as context: 'Answer the question using only the following context: {chunks}. Question: {question}'.

Step 4 — Generate: Azure OpenAI's GPT-4o reads the prompt and generates a precise, grounded answer. Because the answer must come from the chunks, hallucination is minimised.

Step 5 — Cite: The source document name and page number are returned alongside the answer so the user can verify.

!! Key interview concept — why chunks overlap

If you split 'The contract terminates on 30 days notice. This notice must be in writing.' at the full stop, the second sentence loses its context. With 20% overlap, both sentences appear in adjacent chunks. This is called `chunk_overlap` and Infosys interviewers ask about it.

3. Tech Stack & Prerequisites

Azure OpenAI Service	Hosts GPT-4o (generation) and text-embedding-ada-002 (embeddings). You access both via API key. Free tier gives you enough credits to build and demo.
LangChain	Python framework that wires everything together — PDF loaders, text splitters, embedding wrappers, vector store connectors, and QA chains. One library orchestrates the full pipeline.
ChromaDB	Open-source vector database that runs locally. Stores your document embeddings and performs semantic similarity search. Persists to disk so you don't re-embed on every run.
Streamlit	Python library to build web UIs in pure Python — no HTML/CSS/JS needed. You write Python and get a running web app. Infosys uses this for client POC demos.
FastAPI	Optional REST API wrapper. Lets the Streamlit frontend call the backend over HTTP instead of importing it directly. Makes the project look more production-realistic.
Python 3.11+	Core language. All libraries above are Python-native.

requirements.txt

```
# LLM & orchestration
langchain==0.2.16
langchain-openai==0.1.23
langchain-community==0.2.16

# Vector store
chromadb==0.5.5

# PDF processing
pypdf==4.3.1

# Frontend
streamlit==1.38.0

# API layer (optional)
fastapi==0.114.0
uvicorn==0.30.6

# Config
python-dotenv==1.0.1
openai==1.45.0
```

4. Phase 1 — Environment Setup

Azure OpenAI Setup (step-by-step)

1. Go to portal.azure.com and create a free account (you get \$200 credits).
2. Search for 'Azure OpenAI' and create a resource in East US region.
3. Inside the resource, go to 'Model deployments' → Deploy gpt-4o and text-embedding-ada-002.
4. Copy your endpoint URL and API key from the 'Keys and Endpoint' section.
5. Note your deployment names — these are not the model names, they are what you set during deploy.

.env — never commit this file

```
AZURE_OPENAI_API_KEY=your_api_key_here
AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com/
AZURE_OPENAI_API_VERSION=2024-02-01
AZURE_OPENAI_CHAT_DEPLOYMENT=gpt-4o # your deployment name
AZURE_OPENAI_EMBED_DEPLOYMENT=text-embedding-ada-002
```

Project Directory Structure

Folder layout

```
enterprise-doc-chatbot/
.env # Azure keys — add to .gitignore
.gitignore
requirements.txt
README.md # demo GIF + setup instructions
|
src/
  ingest.py # Phase 2: PDF loading + embedding
  vectorstore.py # Phase 3: ChromaDB setup + query
  chain.py # Phase 4: LangChain QA chain
  app.py # Phase 5: Streamlit UI
  api.py # Phase 6: optional FastAPI wrapper
  |
data/
  sample_docs/ # put test PDFs here
  |
  chroma_db/ # ChromaDB persists here (auto-created)
```

5. Phase 2 — PDF Ingestion & Chunking

This is the ingestion pipeline. It reads uploaded PDFs, splits them into chunks, and prepares them for embedding. The chunking strategy directly affects answer quality — too small and you lose context; too large and retrieval becomes imprecise.

`src/ingest.py`

```
import os
from dotenv import load_dotenv
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

load_dotenv()

def load_and_chunk_pdfs(pdf_paths: list[str]) -> list:
    """
    Load one or more PDF files and split into overlapping chunks.
    Returns a list of LangChain Document objects.
    """
    all_docs = []

    for path in pdf_paths:
        loader = PyPDFLoader(path)
        pages = loader.load() # each page = 1 Document
        all_docs.extend(pages)
        print(f'Loaded {len(pages)} pages from {os.path.basename(path)}')

    # Split into chunks
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=500, # ~500 tokens per chunk
        chunk_overlap=100, # 100-token overlap between chunks
        separators=['\n\n', '\n', '.', ' ', ' ', ''],
    )
    chunks = splitter.split_documents(all_docs)
    print(f'Created {len(chunks)} chunks from {len(pdf_paths)} documents')
    return chunks

# Quick test
if __name__ == '__main__':
    chunks = load_and_chunk_pdfs(['data/sample_docs/contract.pdf'])
    print(chunks[0].page_content[:200])
    print(chunks[0].metadata) # {'source': 'contract.pdf', 'page': 0}
```

>> Interview answer: `chunk_size=500, chunk_overlap=100`

If an Infosys interviewer asks 'Why 500 tokens?', say: 'GPT-4o has a 128k context window, but smaller chunks improve retrieval precision. 500 tokens is roughly one paragraph — enough context for most enterprise Q&A; tasks. The 100-token overlap prevents boundary split issues where a key fact straddles two chunks.'

6. Phase 3 — Vector Store with ChromaDB

ChromaDB is the memory of the system. It stores each chunk as a vector (a numerical fingerprint of its meaning) and retrieves the most semantically relevant chunks when a question is asked.

[src/vectorstore.py](#)

```
import os
from dotenv import load_dotenv
from langchain_openai import AzureOpenAIEmbeddings
from langchain_community.vectorstores import Chroma

load_dotenv()

def get_embeddings():
    '''Azure OpenAI embedding model – converts text to vectors.'''
    return AzureOpenAIEmbeddings(
        azure_deployment=os.getenv('AZURE_OPENAI_EMBED_DEPLOYMENT'),
        azure_endpoint=os.getenv('AZURE_OPENAI_ENDPOINT'),
        api_key=os.getenv('AZURE_OPENAI_API_KEY'),
        api_version=os.getenv('AZURE_OPENAI_API_VERSION'),
    )

def build_vectorstore(chunks: list, persist_dir: str = 'chroma_db'):
    '''
    Embed all chunks and store in ChromaDB.
    Persists to disk so you don't re-embed on every run.
    '''
    embeddings = get_embeddings()
    vectorstore = Chroma.from_documents(
        documents=chunks,
        embedding=embeddings,
        persist_directory=persist_dir,
    )
    print(f'Stored {len(chunks)} chunks in ChromaDB at {persist_dir}/')
    return vectorstore

def load_vectorstore(persist_dir: str = 'chroma_db'):
    '''Load an existing ChromaDB – skips re-embedding.'''
    return Chroma(
        persist_directory=persist_dir,
        embedding_function=get_embeddings(),
    )

def search(vectorstore, query: str, k: int = 4):
    '''Return top-k most relevant chunks for a query.'''
    results = vectorstore.similarity_search(query, k=k)
    return results # list of Document objects with .page_content + .metadata
```

7. Phase 4 — LangChain + Azure OpenAI Chain

This is the brain of the system. LangChain wires the vector store retrieval to the Azure OpenAI LLM, injects a custom prompt template, and returns answers with source citations.

[src/chain.py](#)

```

import os
from dotenv import load_dotenv
from langchain_openai import AzureChatOpenAI
from langchain.chains import RetrievalQAWithSourcesChain
from langchain.prompts import PromptTemplate

load_dotenv()

PROMPT_TEMPLATE = '''
You are a document analyst for an enterprise client.
Answer the question using ONLY the context below.
If the answer is not in the context, say 'I could not find this in the provided
documents.'
Always mention which document and page the answer comes from.

Context:
{summaries}

Question: {question}

Answer:'''

def build_chain(vectorstore):
    llm = AzureChatOpenAI(
        azure_deployment=os.getenv('AZURE_OPENAI_CHAT_DEPLOYMENT'),
        azure_endpoint=os.getenv('AZURE_OPENAI_ENDPOINT'),
        api_key=os.getenv('AZURE_OPENAI_API_KEY'),
        api_version=os.getenv('AZURE_OPENAI_API_VERSION'),
        temperature=0, # 0 = deterministic, factual answers
        max_tokens=800,
    )

    prompt = PromptTemplate(
        template=PROMPT_TEMPLATE,
        input_variables=['summaries', 'question'],
    )

    chain = RetrievalQAWithSourcesChain.from_chain_type(
        llm=llm,
        retriever=vectorstore.as_retriever(search_kwargs={'k': 4}),
        chain_type_kwargs={'prompt': prompt},
        return_source_documents=True,
    )
    return chain

def ask(chain, question: str) -> dict:
    result = chain({'question': question})
    return {
        'answer': result['answer'],
        'sources': result['sources'],
        'docs': result['source_documents'],
    }

```

>> **Why temperature=0?**

For enterprise document Q&A, you want factual, deterministic answers — not creative ones. temperature=0 makes the LLM pick the highest-probability token at each step, reducing variability. This is a key design decision: for creative tasks use 0.7+; for document retrieval use 0. Infosys interviewers will ask why you chose this value.

8. Phase 5 — Streamlit Frontend

Streamlit turns your Python backend into a working web app with no HTML. This is the layer that makes the project demo-able — to an interviewer, to a GitHub visitor, and in a GIF for your README.

src/app.py — Part 1: imports & page setup

```
import streamlit as st
import tempfile, os
from ingest import load_and_chunk_pdfs
from vectorstore import build_vectorstore
from chain import build_chain, ask

st.set_page_config(page_title='DocIQ', page_icon='doc', layout='wide')
st.title('DocIQ — Enterprise Document Intelligence')
st.caption('Upload enterprise PDFs and ask questions in plain English.')
```

src/app.py — Part 2: sidebar PDF upload

```
with st.sidebar:
    st.header('Upload Documents')
    uploaded = st.file_uploader(
        'Upload PDFs', type=['pdf'], accept_multiple_files=True
    )
    if uploaded and st.button('Process', type='primary'):
        with st.spinner('Embedding documents...'):
            paths = []
            for f in uploaded:
                tmp = tempfile.NamedTemporaryFile(
                    delete=False, suffix='.pdf'
                )
                tmp.write(f.read())
                paths.append(tmp.name)
            chunks = load_and_chunk_pdfs(paths)
            vs = build_vectorstore(chunks)
            st.session_state['chain'] = build_chain(vs)
            st.session_state['messages'] = []
            st.success(f'Processed {len(uploaded)} file(s). Ready!')
```

src/app.py — Part 3: chat interface

```

if 'messages' not in st.session_state:
    st.session_state['messages'] = []

for msg in st.session_state['messages']:
    with st.chat_message(msg['role']):
        st.write(msg['content'])

if q := st.chat_input('Ask about your documents...'):
    if 'chain' not in st.session_state:
        st.error('Upload documents first.')
    else:
        st.session_state['messages'].append(
            {'role': 'user', 'content': q}
        )
        with st.chat_message('assistant'):
            result = ask(st.session_state['chain'], q)
            st.write(result['answer'])
            if result['sources']:
                st.caption(f'Source: {result["sources"]}' )
            st.session_state['messages'].append(
                {'role': 'assistant', 'content': result['answer']}
            )

```

9. Phase 6 — Deploy & GitHub Portfolio Setup

A project without a GitHub README and demo is invisible. This phase turns your working app into a shortlistable portfolio piece.

.gitignore — critical before first push

.gitignore

```
.env # NEVER push API keys
chroma_db/ # vector store – too large, regenerate locally
__pycache__/
*.pyc
*.pdf # don't commit client documents
.venv/
```

README.md structure that gets noticed

README.md outline

```
# DocIQ – Enterprise Document Intelligence Chatbot

![Demo GIF](assets/demo.gif) <-- record with ShareX or Loom, export as GIF

## What it does
Upload any enterprise PDF (contracts, SOPs, policies) and ask questions
in plain English. Powered by Azure OpenAI GPT-4o + ChromaDB RAG pipeline.

## Architecture
PDF -> LangChain Loader -> Text Splitter -> Azure Embeddings
-> ChromaDB -> Retrieval Chain -> GPT-4o -> Streamlit UI

## Tech Stack
- Azure OpenAI (GPT-4o + text-embedding-ada-002)
- LangChain 0.2 | ChromaDB | Streamlit | FastAPI

## Setup
1. Clone repo
2. cp .env.example .env and fill in Azure credentials
3. pip install -r requirements.txt
4. streamlit run src/app.py

## Key design decisions
- chunk_size=500, overlap=100 for enterprise doc precision
- temperature=0 for deterministic factual answers
- Persistent ChromaDB – no re-embedding on restart
```

Run locally

Terminal commands

```
# Install
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Run Streamlit app
streamlit run src/app.py

# App opens at http://localhost:8501
# Upload a PDF, type a question, get a cited answer
```

10. Resume Bullet & Interview Q&A;

Exact Resume Bullet to Add

>> Add this under Projects

Enterprise Document Intelligence Chatbot (Azure OpenAI · LangChain · ChromaDB · Streamlit) \n• Built a multi-document Q&A; system over enterprise PDFs using a RAG pipeline — LangChain + ChromaDB for semantic retrieval and Azure OpenAI GPT-4o for cited answer generation. \n• Implemented RecursiveCharacterTextSplitter with 500-token chunks and 100-token overlap to optimise retrieval precision across contract and compliance documents. \n• Delivered a Streamlit interface enabling plain-English queries with source-cited responses, deployable as an enterprise POC.

5 Questions Infosys Will Ask — With Answers

Q1: What is RAG and why did you use it instead of fine-tuning?

RAG retrieves relevant context at inference time from your own documents, making answers accurate and up-to-date without expensive retraining. Fine-tuning bakes knowledge into model weights — it can't update when documents change and costs thousands of dollars for GPT-4o. RAG is cheaper, faster to deploy, and auditable — which is what enterprise clients need.

Q2: Why ChromaDB and not Pinecone or Weaviate?

ChromaDB is open-source and runs locally with zero infrastructure cost — perfect for a POC or on-premise enterprise requirement. Pinecone is managed and cloud-hosted, which adds latency and cost for smaller deployments. For a production rollout, I would evaluate Pinecone for scale.

Q3: How do you handle documents with tables or images?

PyPDFLoader extracts text only. For tables, I would switch to pdfplumber which preserves table structure. For image-heavy documents, I would add pytesseract OCR as a pre-processing step. This is a known limitation of the current implementation.

Q4: How would you scale this to 10,000 documents?

ChromaDB persists to disk but becomes slow at large scale. I would migrate the vector store to Azure AI Search or Pinecone, add a document metadata filter layer so retrieval searches only relevant document subsets, and run embedding in batches via async workers.

Q5: How do you prevent hallucinations?

Three controls: (1) temperature=0 for deterministic outputs, (2) the prompt explicitly says 'answer using ONLY the context below — if not found, say so', and (3) source citations are returned alongside every answer so users can verify. The LLM cannot answer from training knowledge because the context is the only information provided in the prompt.

✓ Final checklist before applying

1. GitHub repo is public with a clear README \n2. .env is in .gitignore — API keys are not exposed \n3. A demo GIF is embedded in the README \n4. You can run the app locally and show it in a screen share \n5. You can answer all 5 questions above without hesitation \n6. The project bullet is on your resume with Azure OpenAI named explicitly